



PHP conditional statements 45m

'if...else'

One of the most used programming control statements is the conditional `if...else` statement. This means the flow is directly based on the state of a conditional expression. If the condition is true, it follows the `if` block; otherwise, if the condition is false, then it follows the `else` block.

The syntax for an **if...else** statement is:

```
if (condition) {  
  
    code to be executed if condition is true;  
  
} else {  
  
    code to be executed if condition is false;  
  
}
```

The example below uses a literal Boolean value to demonstrate the `if...else` syntax. If the condition is true (that is, `$i` is less than 15), the `if` block is executed; otherwise, if the condition is false, the `else` block is executed.



[Skip to main content](#)



```
1  $i = 13;
2      if($i < 15)
3      {
4          echo "The condition was true";
5      }
6  //this block is optional when there is no alternative action
7      else
8      {
9          echo "The condition was false";
10     }
```



The 'if...else' code has been copied into JDoodle. Execute this code to see what is returned. What do you expect the result to be?

Once you've executed the code, have a go at editing the code so that the `if` condition is false and the `else` block gets executed.



ENTER FULLSCREEN

[Skip to main content](#)

^ EXECUTE MODE, VERSION, INPUTS & ARGUMENTS

▼

Interactive
Mode: On

RESULT



Skip to  main content

if...else' loop

Non-Assessed

Write a PHP **'if...else'** statement that tests whether a person is old enough to vote.

Display a message based on their age to let them know that either they are eligible to vote or that they have X (number of) years until they are eligible to vote.

☐ I have completed the task

Submit



Extending the 'if...else' statement

The `if` statement can be nested or compounded, so it is not limited to just a single statement. Nesting `if...else` statements is where you insert `if` statements inside each other so that different conditions can be evaluated one after the other. The `if` can be nested either inside another `if` or inside an `else`. For example:

```
if (TRUE) {  
  
    if (TRUE) {  
  
        ... //code statements  
  
    } else {  
  
        ...  
    }  
}
```

[Skip to main content](#)

```
}  
  
... //code statements  
  
};
```

A compounded `if` uses one or more `elseif` statements instead of a single `else`. Like a nested `if`, different conditions can be evaluated one after the other, but in this case only where the first condition is false. This is used when there are more than one `if...else...` statements to be used. The `elseif` is used to minimise the unnecessary nested statements from many `if...else...`

For example:

```
if (TRUE) {  
  
... //code statements  
  
} elseif (TRUE) { //note the slight variation to the else and if  
pair – one word  
  
... //code statements  
  
} elseif...
```

'switch' statements

The `switch` control structure can evaluate multiple values for the same variable and provide different responses for each value. The variable value can be either a string or a number. Control drops through from any selected value to all those that follow unless you add a **break** statement at the end of each option. These control structures are suitable for menus, command instructions instead of using a series of `if...else...` or `if elseif...else if` statements. It makes for cleaner code.

The example below uses the `rand` function to generate a random number in the range 0–2, then echoes either 'Rock', 'Paper' or 'Scissors', depending on the value.

Skip to main content



ENTER FULLSCREEN

^ EXECUTE MODE, VERSION, INPUTS & ARGUMENTS



Interactive
Mode: On

RESULT

Skip to main content



Thoughts on this page?

